L2: The elements of a Fully Connected Neural Network

EARTHLAB 2026

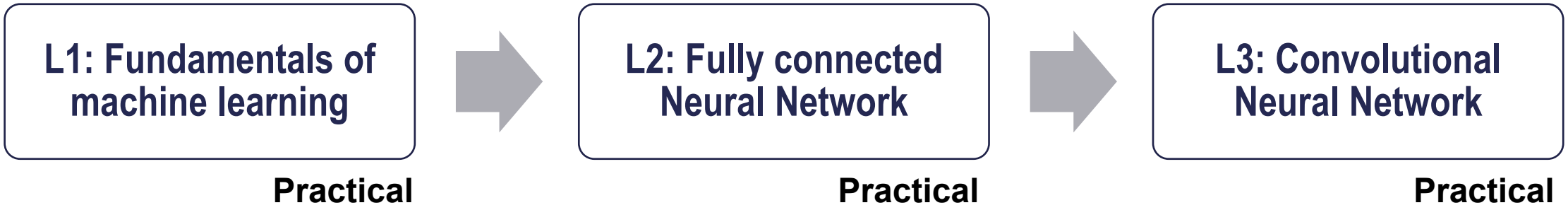
Lecturer: Alice Cicirello (ac685@cam.ac.uk)

No AI was involved in creating these slides ☺

Learning objectives of this lecture

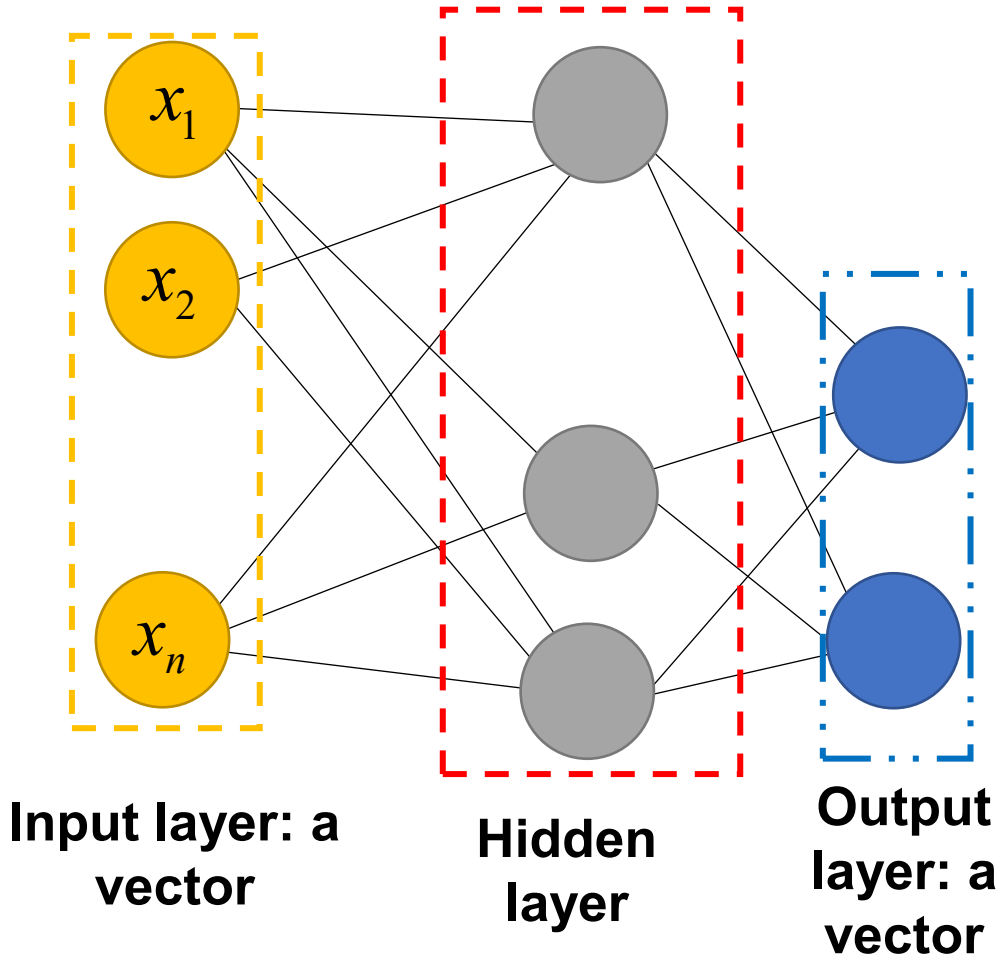
L2: The elements of a Fully Connected Neural Network (FCNN)

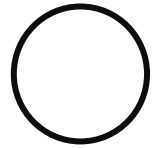
- Understand the **elements** of a Perceptron and of a Multi-Layer Perceptron (aka Feed Forward NN)
- Understand **how to setup** a Fully Connected Neural Network through 5 steps: architecture setup, optimization setup, training, validation and testing
- Understand **practices in applied machine learning**: k-fold cross-validation and regularization
- Understand how FCNN are part of **SOTA architectures**



Why are ARTIFICIAL NEURAL NETWORK (ANN) so important?

- ANN are general function approximations!
- They can be applied to almost any ML problem where the problem is about learning a complex mapping from the input to the output space (when large and informative data is available!)
- The elements of the Fully Connect NN are at the core of the deep-learning and more complex architectures





The neuron!

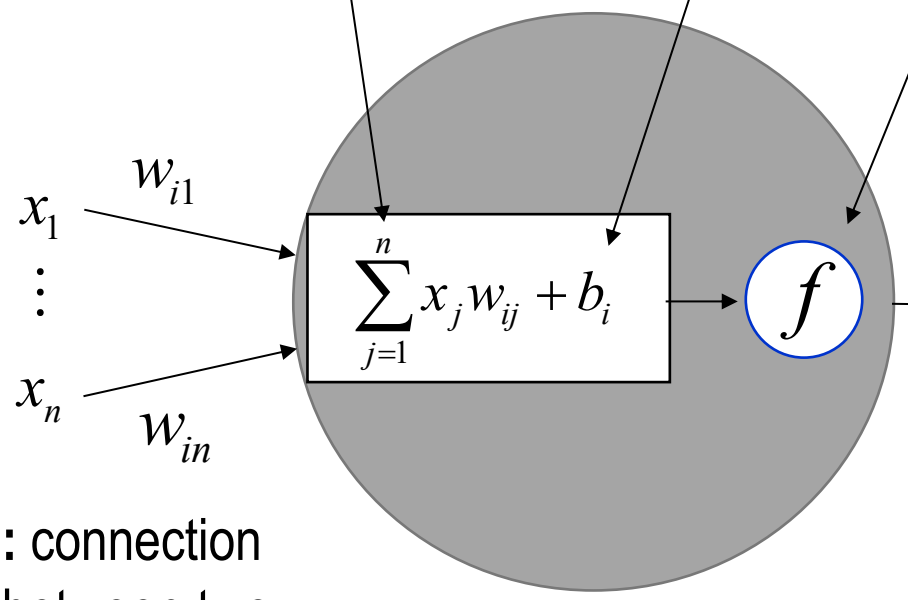
A close look inside a **node: the perceptron – the computational unit** (1957!)

Bias: shift the value, to add more flexibility in modelling complex patterns

Linear combination

Activation function: **introduce non-linearity in the network**

Inputs: **real valued vector**
(each entry would correspond to a node)



output i
numerical values

Weights: connection strength between two nodes ij: scalar quantity

Neuron i

$$y_i = f\left(\sum_{j=1}^{n} w_{ij} x_j + b_i\right)$$

i =index of the neuron in the layer
j= **input index** in the NN
w scalar weights

Perceptron is the simplest **Neural Network (NN)** architecture… made by a **single neuron**!
It can learn linearly separable functions or decision boundaries – **how?**
By learning the weights and biases that minimize an error metric

Learning the weights and bias as an **optimization problem**

$$y_i = f\left(\sum_{j=1}^{n} w_{ij} x_j + b_i\right)$$

A parametric nonlinear function (**the weights and biases are the parameters!**) from the input variables **x** to the **outputs y_i**

- Given a training **D**ataset of **size N** with pairs of **inputs** x_j **and target outputs** z_j $\quad D = \{\mathbf{X}, \mathbf{Z}\}$
- Train the model (i.e. obtain the weights and biases!) so that its prediction produces a **small error** wrt the target outputs. A simple choice of **error (E)** and **(total) loss function (L)**:

$$E_i = \frac{1}{2}\left(z_i - y_i(\mathbf{w})\right)^2 \qquad L(\mathbf{w}, b) = \frac{1}{2N}\sum_{i=1}^{N}\left(z_i - y_i(\mathbf{w}, b)\right)^2 = \frac{1}{N}\sum_{i=1}^{N} E_i$$

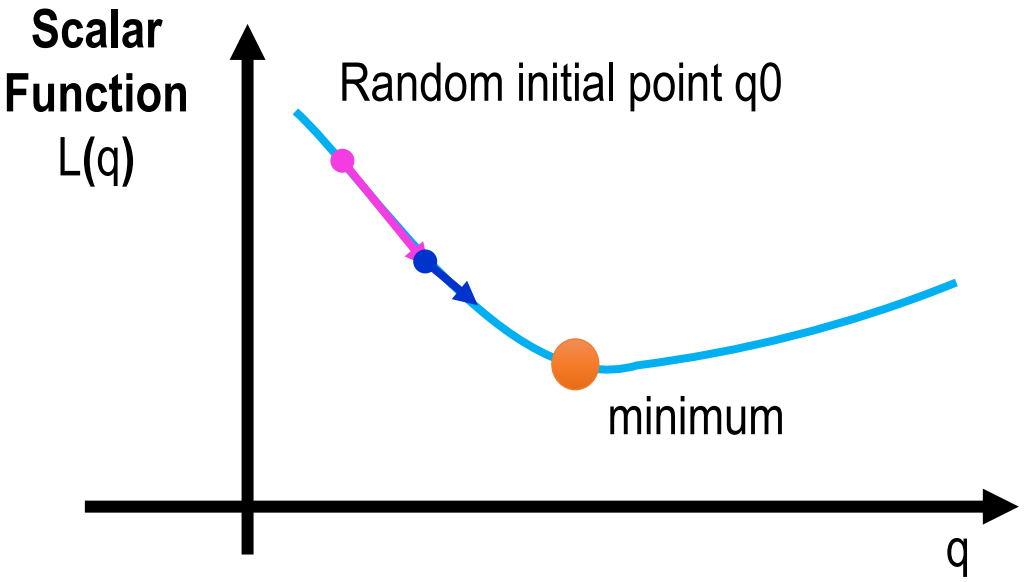
- Cast the problem as an optimization:

$$(\mathbf{w}^*, b^*) = \arg\min_{\mathbf{w}, b} L(\mathbf{w}, b) = \frac{1}{2N}\sum_{i=1}^{N}\left(z_i - y_i(\mathbf{w}, b)\right)^2 \Rightarrow \nabla_{\mathbf{w},b} L(\mathbf{w}, b) = 0$$

Gradient

Solving **an optimization** with **gradient descent: example in one dimension**

Gradient Descent: iterative method used to find minima of a function $\nabla_q L(q) = 0$



- Choose a random point (in the **Stochastic** GD!)
- Follow the **negative** gradient to find the minima
- The **larger the magnitude of the gradient is, the more we move**. We slow down closer to a minimum
- **The learning rate, alpha,** is introduced so that the parameter q is not updated by the full amount, and slow-down is controlled.
- At each iteration t, learn the parameter q as:

$$q(t) = q(t-1) - \alpha \frac{\partial L(q)}{\partial q}$$

Solving **the weights and bias optimization** problem with **gradient descent**

$$\nabla_{\mathbf{w},b} L\left(\mathbf{w},b\right)=0$$

- finding the best set of weights **w and bias** that perfectly minimize the loss function is a **very difficult** (or even impossible problem as the number of weights increases – especially for complex NN)
- **refining** a specific set of weights **w and bias** is slightly better and significantly less difficult.
  → will start with a random **w** and then iteratively refine it, making it slightly better each time!

$$w_{ij}\left(t\right)=w_{ij}\left(t-1\right)-\alpha\frac{\partial L\left(\mathbf{w},b\right)}{\partial w_{ij}}$$

$$b\left(t\right)=b\left(t-1\right)-\alpha\frac{\partial L\left(\mathbf{w},b\right)}{\partial b}$$

How do we compute the gradients??

When to stop?

- when **w** and b reach convergences → ideally!
- when the loss almost convergence → in reality

Back-propagation: algorithm for calculating gradients (*rediscovered in the 1980s)

Backprop "is an algorithm that computes the chain rule, with a specific order of operations that is highly efficient" (Page 205, Deep Learning, 2016.)

$$w_{ij}(t) = w_{ij}(t-1) - \alpha \frac{\partial L(\mathbf{w},b)}{\partial w_{ij}} \qquad b(t) = b(t-1) - \alpha \frac{\partial L(\mathbf{w},b)}{\partial b}$$

We know

$$y_i = f\left(\sum_{j=1}^{n} w_{ij} x_j + b_i\right)$$

$$L(\mathbf{w},b) = \frac{1}{2N}\sum_{i=1}^{N}\left(z_i - y_i(\mathbf{w},b)\right)^2 = \frac{1}{N}\sum_{i=1}^{N} E_i$$

Chain rule!

$$\frac{\partial L(\mathbf{w},b)}{\partial w_{ij}} =$$

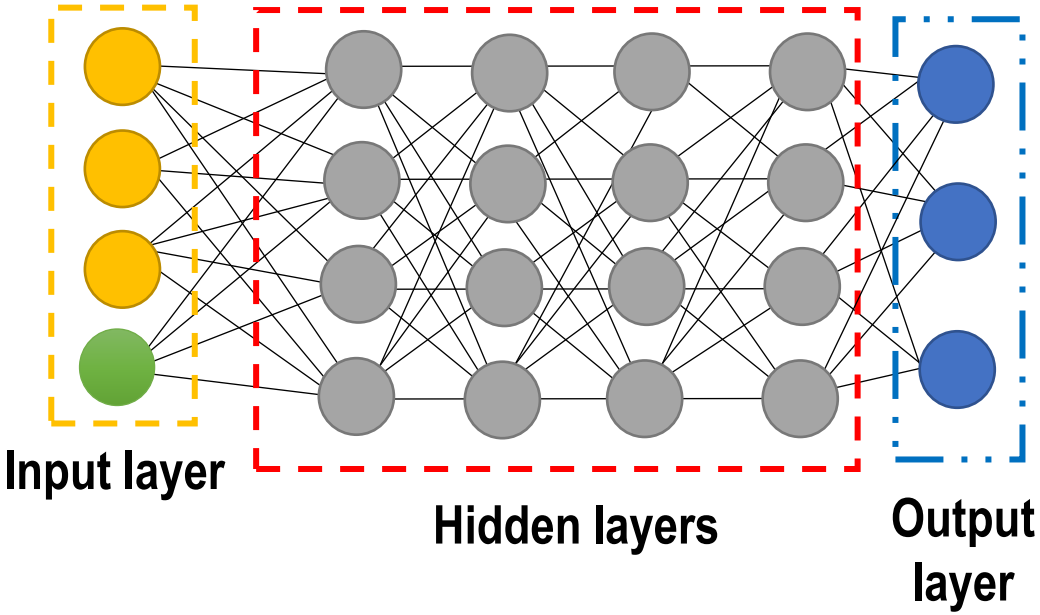
$$w_{ij}(t) = w_{ij}(t-1) - \alpha f' \delta_i x_j \qquad b(t) = b(t-1) - \alpha f' \delta_i$$

Learning the weights and bias (note: in SGD N=1!):

- Set initial random weights and bias, and carry out the "**forward propagation**"
- **Backprop** to learn the new set of weights and bias

The Multi-Layer Perceptron (MLP)*: a cascade of single-layer perceptrons

*** Also known as Feed Forward Neural Network**



- **Input Layer**: Input variables, sometimes called the visible layer represented as a single vector
  → usually not counted when numbering the layers!!!
  → inputs can come from different sources (e.g. distinguished as yellow and green)
- **Hidden Layers**: Layers of nodes between the input and output layers. There may be one or more of these layers.
- **Output Layer**: A layer of nodes that produce the output variables.

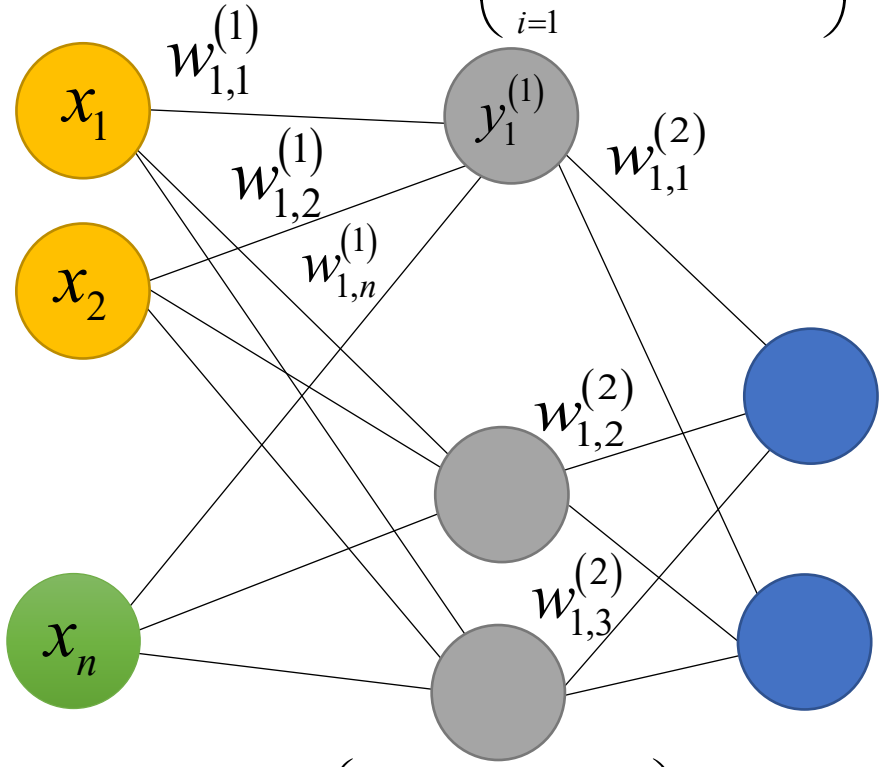
- **Densely or Fully connected Neural Network (<u>our focus!</u>):**
  - each neuron is fully connected to all neurons in the previous and subsequent layer
  - neurons in a single layer are completely independent and do not share any connections
- The NN can solve complex non-linear problems depending **on the data and network architecture**
- A **model's capacity** is its **ability to fit a wide variety of functions**: we can control whether a model is more likely to overfit or underfit by altering its capacity.

*model capacity: what a model can represent. Model complexity: what it does represent (e.g. number of free parameters). A complex model has high capacity, and liis kely to overfit

Fully Connected NN: computing the output with one hidden layer

- **Hierarchy of layers** builds up increasing levels of abstraction from the input to the output variables
- For each layer: activation function (which we select), **weights, and a bias → we must learn them!**

$$y_1^{(1)} = f_h\left(\sum_{i=1}^{n} w_{1i}^{(1)} x_i + b_1^{(1)}\right)$$



$$y_3^{(1)} = f_h\left(\sum_{i=1}^{n} w_{3i}^{(1)} x_i + b_3^{(1)}\right)$$

One node

$$y_j = f\left(\sum_{i=1}^{n} w_{ji} x_i + b_j\right)$$

MLP with one hidden layer

$$y_k = f_{output}\left(\sum_{j=1}^{m_h} w_{kj}^{(2)} f_h\left(\sum_{i=1}^{n} w_{ji}^{(1)} x_i + b_j^{(1)}\right) + b_k^{(2)}\right)$$

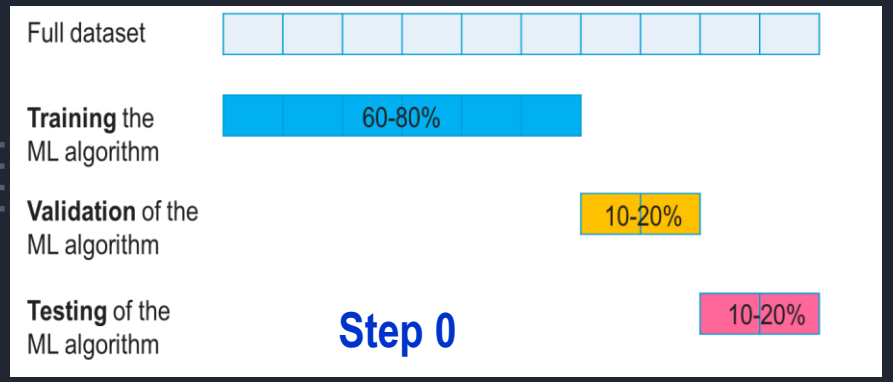
$$y_1 = f_{output}\left(\sum_{j=1}^{m_h=3} w_{1j}^{(2)} y_j^{(1)} + b_1^{(2)}\right)$$

$$y_2 = f_{output}\left(\sum_{j=1}^{m_h=3} w_{2j}^{(2)} y_j^{(1)} + b_2^{(2)}\right)$$

Similarly, MLP with two hidden layers

$$y_r = f_{output}\left(\sum_{k=1}^{m_{h2}} w_{rk}^{(3)} f_{h_2}\left(\sum_{j=1}^{m_{h1}} w_{kj}^{(2)} f_{h_1}\left(\sum_{i=1}^{n} w_{ji}^{(1)} x_i + b_j^{(1)}\right) + b_k^{(2)}\right) + b_r^{(3)}\right)$$

(*) indicates the layer number, not an exponent



Steps for building a fully connected Neural Network (NN) model

- **Step 1: NN architecture setup**
- **Step 2: Optimization setup**
- **Step 3: Network training**
- **Step 4: Tuning of Network parameters on validation data set**
- **Step 5: Test on the NN**

Step 1: NN architecture setup

This step controls the types of **mapping functions** that a Neural Network is able to learn.

Architecture: the specific arrangement of the layers and nodes in the network

(i) number of nodes of the output layer → **Width**

(ii) number of nodes of the input layer; → **Width**

(iii) number of nodes of the hidden layer; → **Width**

Size: total number of nodes

(iv) number of hidden layers;

Depth: total number of hidden layers +output layer

(v) activation function for the hidden layers;

(vi) activation function for the output layer;

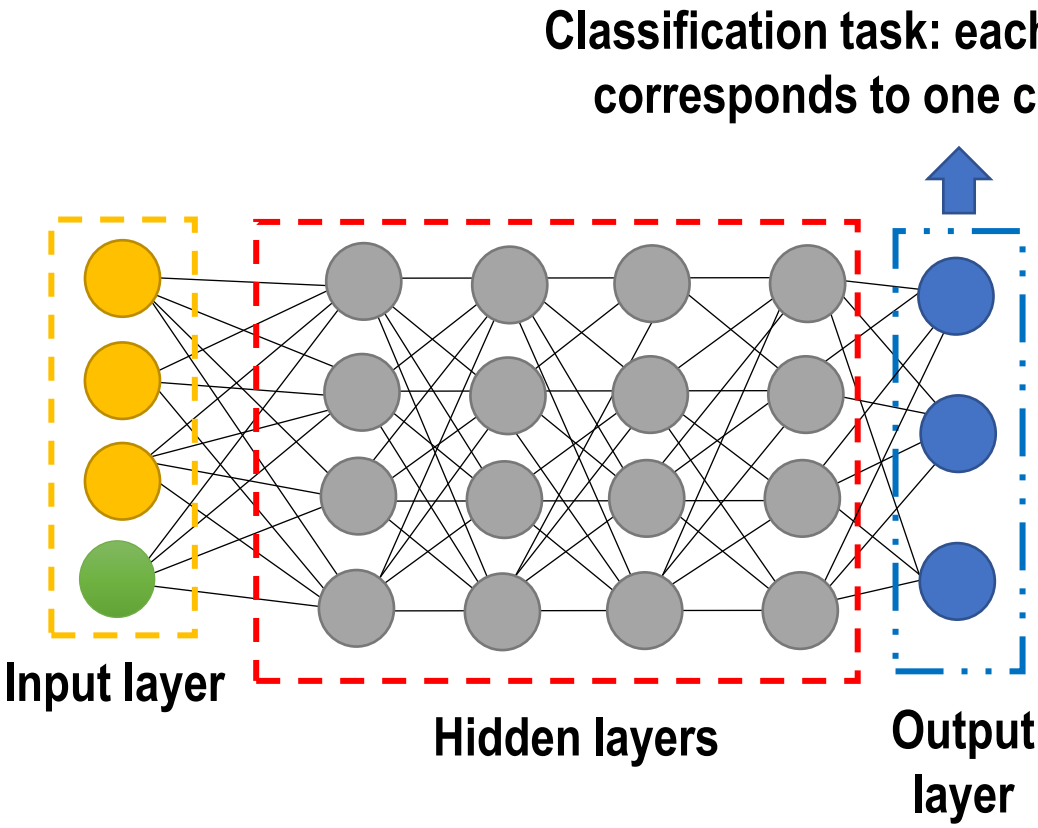
(vii) performance metric: loss function.

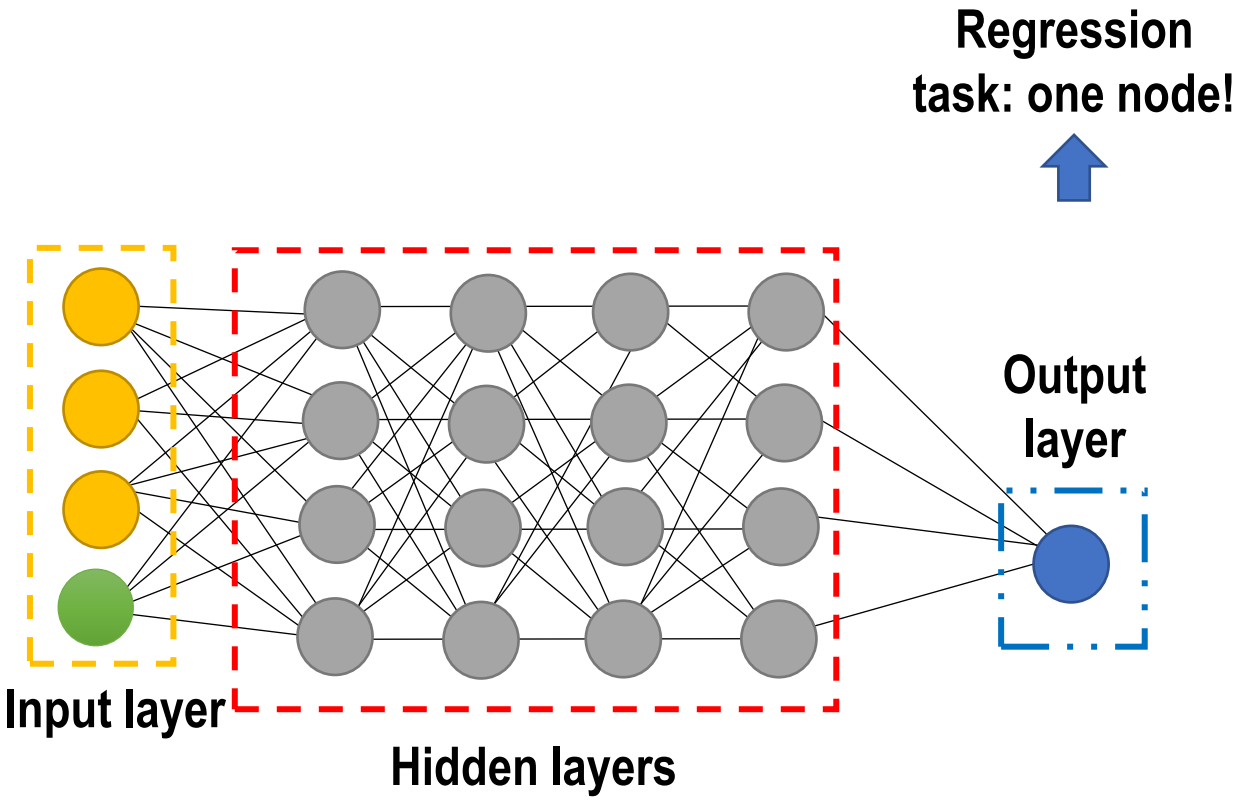
Note: (iii)-(v) need to be specified at first. They are tuned subsequently in step 4

This initial step is very important: setting up wrongly, could drastically affect **the model capacity** of learning the training dataset (which can be controlled by changing width and depth), and as well as **computational cost** of identifying a model that provides a good fit!

Step 1: NN architecture setup

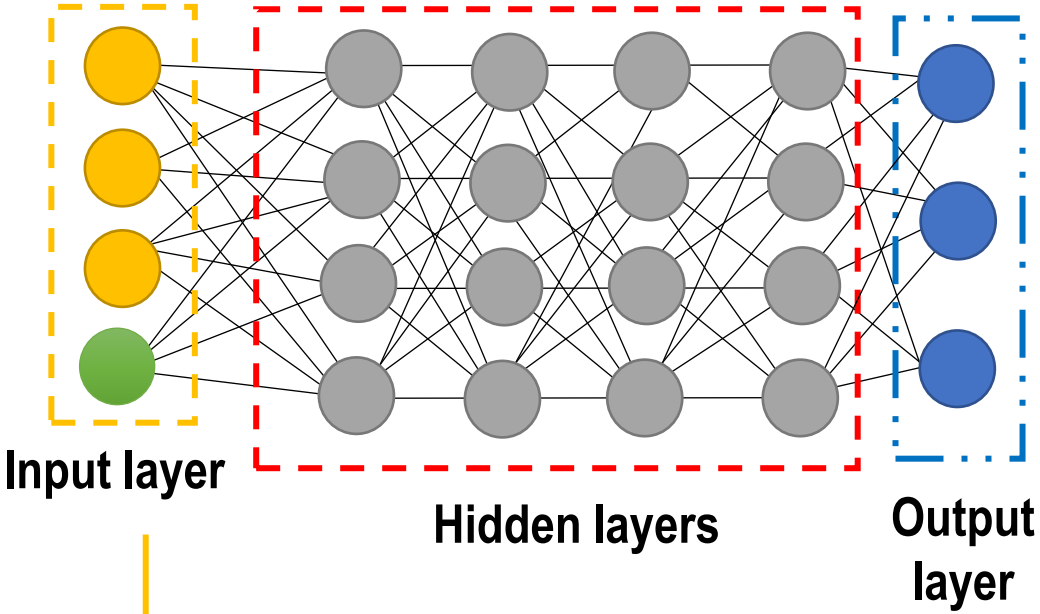
Nodes in the Output Layers: depend on task

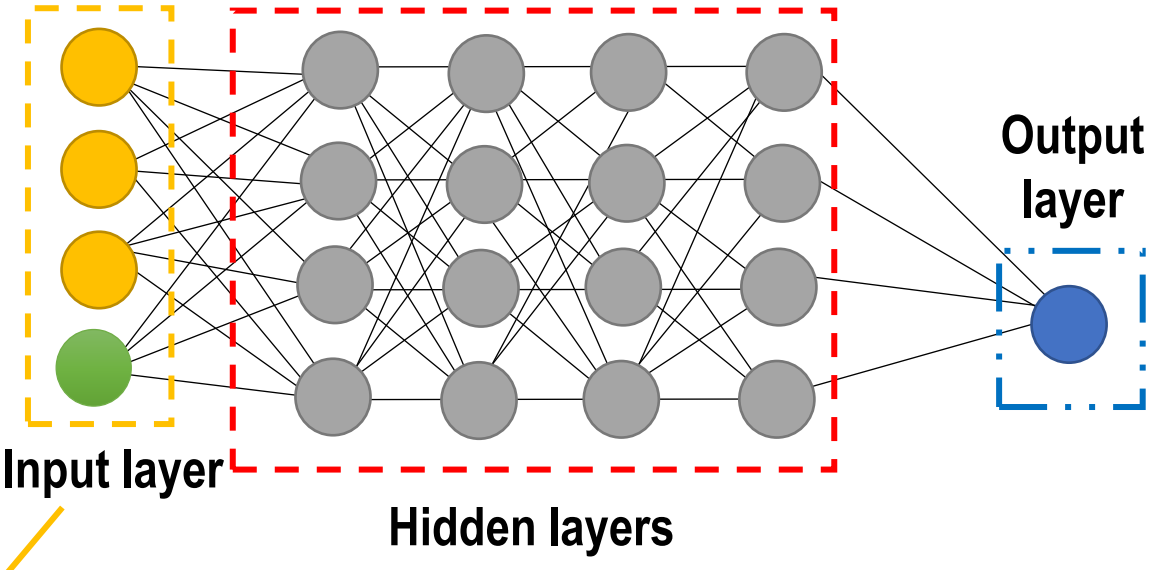




Step 1: NN architecture setup

Nodes in the Input Layers: depend on the dataset





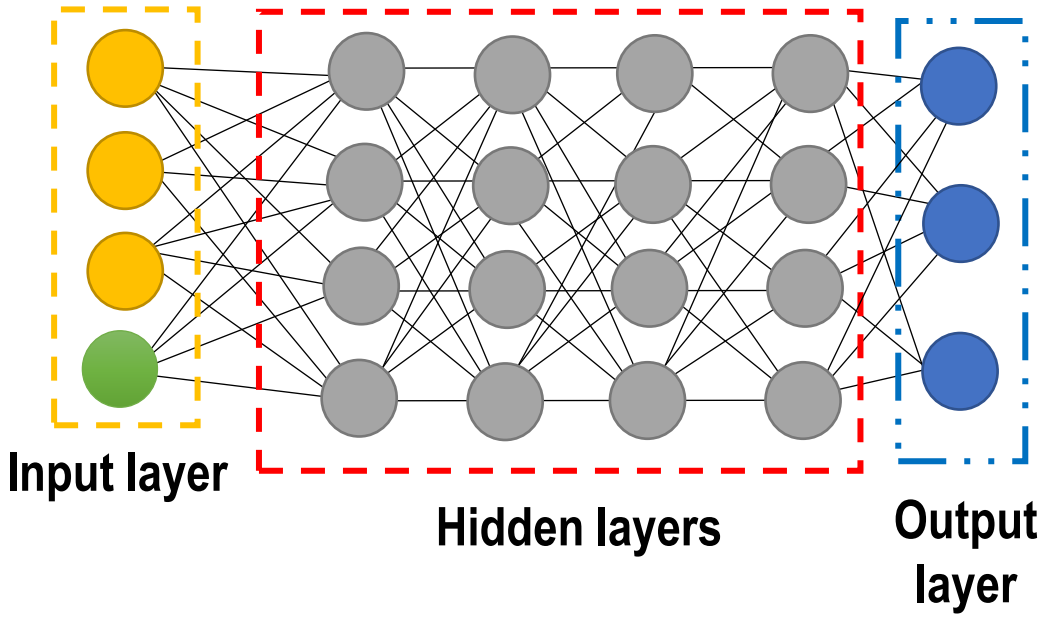
The "visible layer": (multi-modal) raw data or features collected in a single vector

How many nodes? Number of inputs/known features

- Carefully selecting the features to reduce the complexity of the data (make patterns more visible) can prevent a model from becoming too specific to the training data set (**overfitting**)
- **Informative data** is better than using a large amount of uninformative data!

Step 1: NN architecture setup

Hidden layers: where the data-processing happens!



How many layers? No rule-of-thumb!

How many neurons per hidden layer?
a number between the size of the input layer and the size of the output layer.

- **small number** of neurons can lead to **underfitting** and high statistical bias.
- **too many** neurons may lead to **overfitting**, high variance, and increases the time it takes to train the network.

Note:

- **Starting set up** for many problems: 1 hidden layer, number of neurons equal to the mean number of neurons of the input and output layer.
- **Pruning**: trimming neurons which have no impact on the performance of the network during training (weights of the neurons that are close to zero are removed).
- Number of nodes and hidden layers are architecture parameters to be **tuned** (step 4 – during validation)

Step 1: NN architecture setup

Why the **number of hidden layers** matters?

Table 5.1: Determining the Number of Hidden Layers

Number of Hidden Layers	Result
none	Only capable of representing linear separable functions or decisions.
1	Can approximate any function that contains a continuous mapping from one finite space to another.
2	Can represent an arbitrary decision boundary to arbitrary accuracy with rational activation functions and can approximate any smooth mapping to any accuracy.

Table from an Introduction to Neural Networks for Java, Second Edition

MLPs are **universal approximators (interesting reads!)**:

- *"Specifically, the universal approximation theorem states that a feedforward network with a linear output layer and at least one hidden layer with any "squashing" activation function (such as the logistic sigmoid activation function) can approximate any Borel measurable function from one finite-dimensional space to another with any desired non-zero amount of error, provided that the network is given enough hidden units."*
  Page 198, Deep Learning, 2016.
- *"that shows that an MLP with two hidden layers is sufficient for creating classification regions of any desired shape. This is instructive, although it should be noted that no indication of how many nodes to use in each layer or how to learn the weights is given."*
  Lippmann in the 1987 paper "An introduction to computing with neural nets

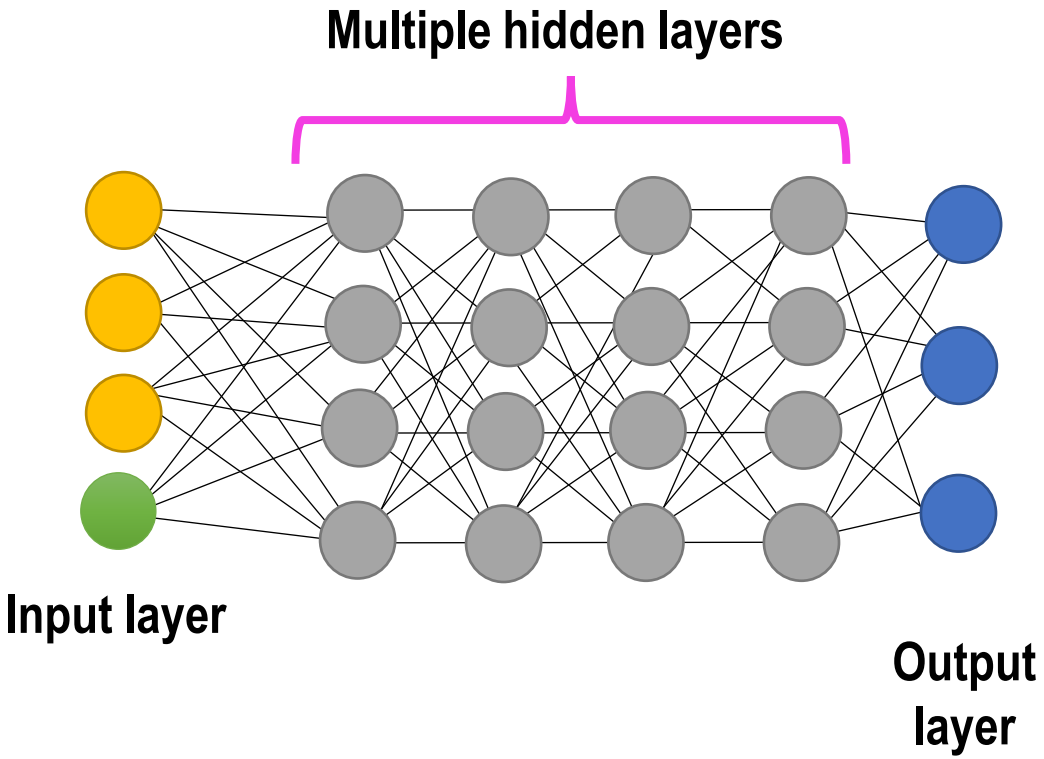
Step 1: NN architecture setup

When the number of hidden layers is large.. **Deep Neural Networks**!

- **What is DEEP? more than 1 hidden layer!**

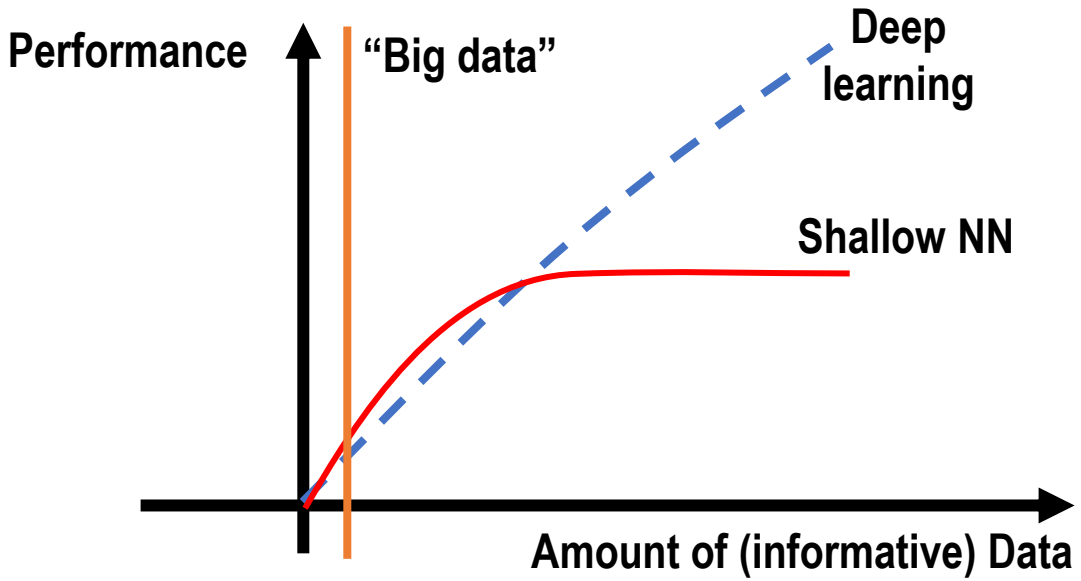
- **What are the advantages of Deep NN?**
  - "By adding more layers and more units within a layer, a deep network can represent functions of increasing complexity." Page 167, Deep Learning, 2016.
  - learn high-level features from data in an incremental manner
  - eliminates the need of domain expertise and feature engineering

- The deep learning problem is about "discovering a set of underlying factors of variation that can in turn be described in terms of other, **simpler underlying factors of variation**." Page 201, Deep Learning, 2016



Step 1: NN architecture setup

Deep NN vs shallow NN architectures



- **When to use it?** If the function to be learnt involve the composition of several simpler functions.
- **How do we know in advance? We don't!** it requires experience with the domain and with modeling problems with NN

- Typical applications (which require more complex NN architectures than FCNN):
  - image classification,
  - natural language processing
  - speech recognition

- **Needs big data**!!! Need high-end computational infrastructure;
- **Lacks Interpretability**: does not reveal why the model succeeds/fails (note: even if we can understand which nodes of a deep NN were activated, we would not know what these layers of neurons were doing).

Step 1: NN architecture setup

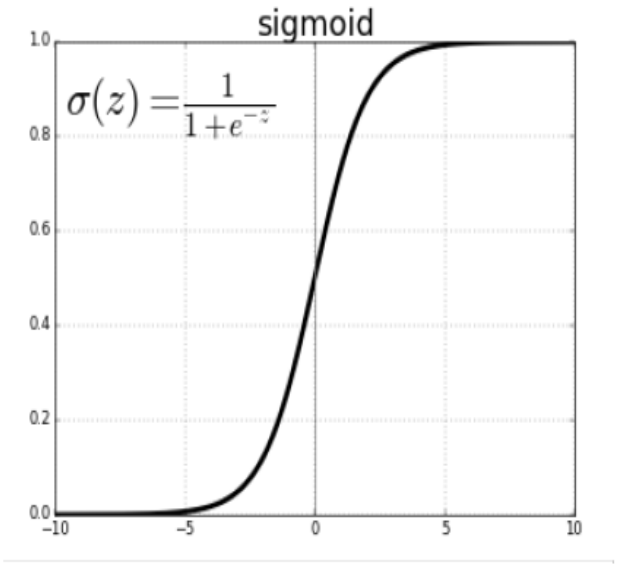
Activation function: how the weighted sum of the input in a node is transformed into an output.

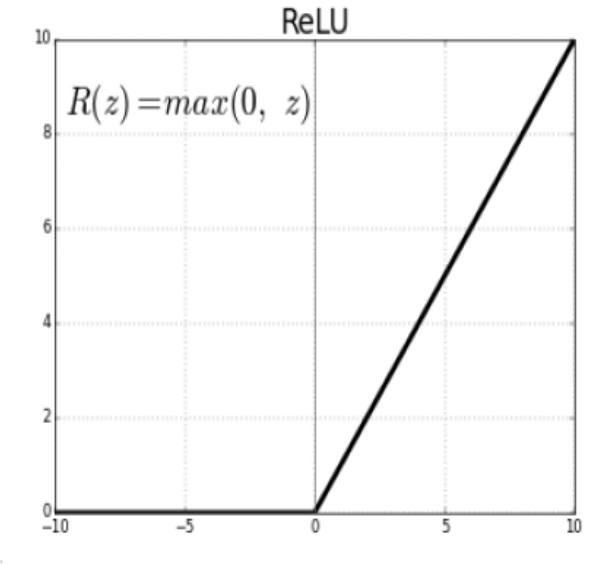
- Activation function are also called *transfer function* or *squashing function*.
- Many activation functions are:
  - **nonlinear** and may be referred to as the "*nonlinearity*" in the layer or the network design.
  - **differentiable**, meaning the first-order derivative can be calculated for a given input value → important for backpropagation.
- Normally they can be grouped into

Continuous

Discontinuous

takes any real value z as input and outputs values in the range 0 to 1



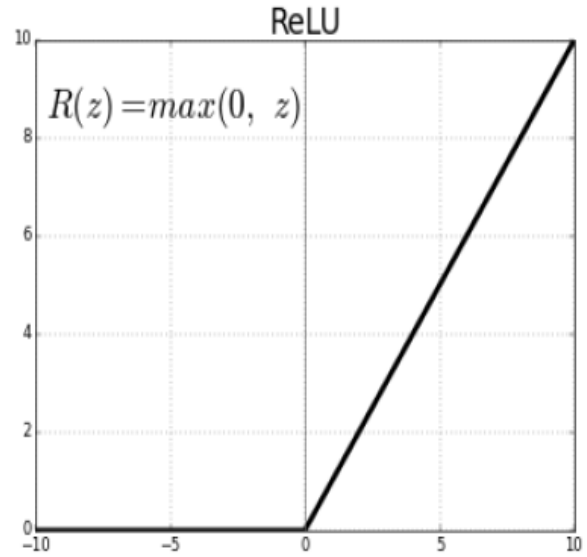


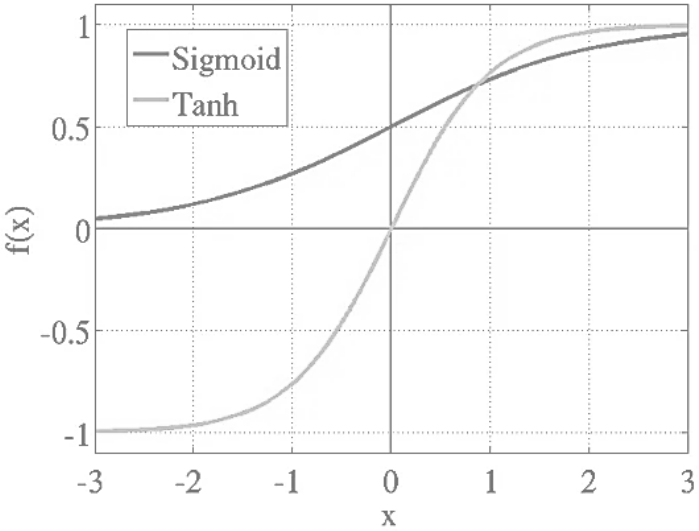
if the input value (z) is negative, then a value 0.0 is returned, otherwise, the value is returned.

Step 1: NN architecture setup

Typical differentiable (wrt the weights) nonlinear **activation functions for hidden layers**

- Rectified Linear Activation (**ReLU**)
  - most common, use this in the first instance for MLP and Convolutional NN
  - **less susceptible to vanishing gradients that prevent deep models from being trained,** it can suffer from other problems like saturated or "dead" units
  - Good practice:
    - **weight initialization:** "He Normal" or "He Uniform"
    - **scale input data prior to training:** to the range 0-1 (normalize).
- Logistic (**Sigmoid**) – used in Recurrent NN
  - Good practice:
    - **weight initialization:** "Xavier Normal" or "Xavier Uniform"
    - **scale input data prior to training**: to the range 0-1 (normalize)
- Hyperbolic Tangent (**Tanh**) – used in Recurrent NN
  - very similar to the sigmoid - has the same S-shape.
  - Takes any real value as input and outputs values in the range -1 to 1.
  - Good practice:
    - **weight initialization:** "Xavier Normal" or "Xavier Uniform"
    - **scale input data prior to training:** to the range -1 and 1.

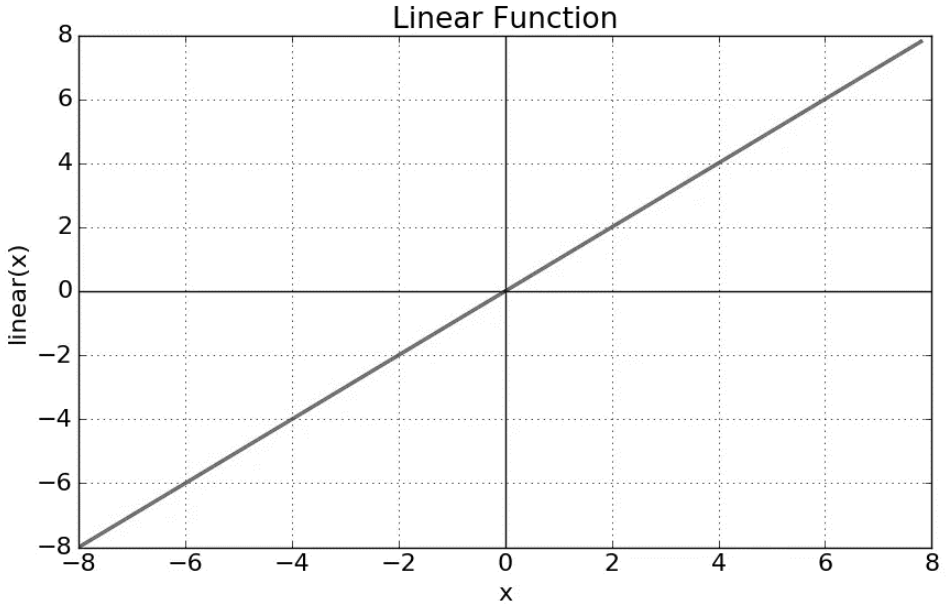




Step 1: NN architecture setup

Activation function for the output layer: define the type of predictions the model can make

- **Linear** (also known as identity of "no activation")
  - For **regression** tasks
  - Returns the value directly
  - Good practice:
    - model with a linear activation function in the output layer are typically scaled prior to modeling using normalization or standardization transforms



- Logistic (**Sigmoid**) – same as in the hidden layers
  - For **Binary Classification** (One node), **Multilabel Classification** (One node per class)
  - Target labels used to train a model with this AF in the output layer will have the values 0 or 1.

- **Softmax** (cannot be shown into a graph)
  - For **Multiclass Classification** (One node per class)
  - outputs a vector of values that sum to 1.0 that can be interpreted as probabilities of class membership.
  - Target labels used to train a model with this AF in the output layer will be vectors with 1 for the target class and 0 for all other classes.

Step 1: NN architecture setup

- Note: in the space of possible sets of weights, find the model that will give **good or good enough predictions** - since there are too many unknowns, we cannot aim to calculate the perfect weights!
- **Implementation**: we seek to **minimize the error**. The objective function of this optimization problem is referred to as a **loss function** (or just **loss**).
- Depending on the task:
  - **Regression: mean squared (MSE) error**
  - **Classification: binary or multi-class cross-entropy error**
- Typically, the error function is nonlinear and it might display multiple local minima. → **iterative optimization procedure**

Learning the weights as an optimization problem using a **loss function**

$$\mathrm{MSE} = \frac{1}{N}\sum_{k=1}^{N}\left(y_k - z_k\right)^2$$

$$\mathrm{L}\left(p,q\right) = -\sum_{c=1}^{Classes} p\left(z_c\right)\log \underbrace{q\left(y_c\right)}_{\substack{\text{model predicted prob}}}^{\text{true prob}}$$

Steps for building a fully connected Neural Network model

Step 1: NN architecture setup

Step 2: Optimization setup

Step 3: Network training

Step 4: Tuning of Network parameters on validation data set

Step 5: Test on the NN

Step 3: Network training

Choosing a training configuration (i.e. optimization setup): using the training data & iterative search

Mapping mode: forward propagation

Calculations flow from the input layer to the output layer initial designed

The later layers **give no feedback** to the previous layers.

The loss function is used to assess the prediction error

Learning phase: backward propagation

Two stages can be distinguished:

1. the derivatives of the error function with respect to the weights and biases are computed
2. these are used to determine the adjustment to be made on the weights and biases.

These derivatives are usually computed with the **backpropagation** technique, or more advance techniques such as **Automatic Differentiation**, which embed backprop

Parameters of the optimization setup (optimization algorithm, learning rate, epochs, batch size and weights initialisation) can be adjusted to **reduce the total loss**

Iterations are continued until a sufficient accuracy is achieved or computational budget is used up

Step 2: Optimization setup

Batch size and Epochs

- **Batch size**: number of training samples to work through before the error is calculated and the model's internal parameters are updated.
- **Number of epochs:** controls the number of complete passes through the training dataset – one epoch is comprised of one or more batches.
  - One epoch: each sample in the training dataset has had an opportunity to update the internal model parameters.
  - The number of epochs is traditionally large, often hundreds or thousands. **It is an integer number between one and infinity that is independent on the training data size.** This is to allow the learning algorithm to run until the error from the model has been sufficiently minimized.

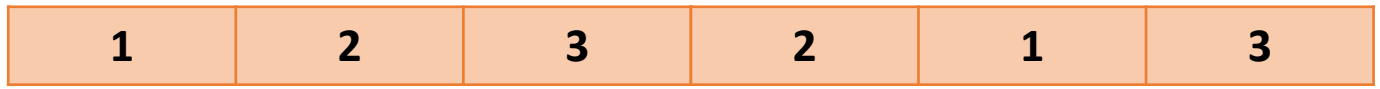
Full dataset: 10 samples

Training dataset: 6 samples

Example of Batch size=2 (2 samples) → 3 Batches

Example of Epochs=1 (comprised of 3 Batches)

Example of Epochs=2 (comprised of 6 Batches) – **aka 2 iterations** of the training dataset



Step 2: Optimization setup

Learning algorithms: SGD (Stochastic Gradient Descent) and BGD (Batch Gradient Descent)

Stochastic Gradient Descent (SGD)

Batch size: 1 sample

- **Frequent updates:**
  - Will give immediate insight on performance and rate of improvement
  - But can result in a noisy gradient signal → this might cause the model parameters and in turn the model error to jump around
- **It is more computationally expensive for large datasets**

Batch Gradient Descent (BGD)

Batch size: all training samples

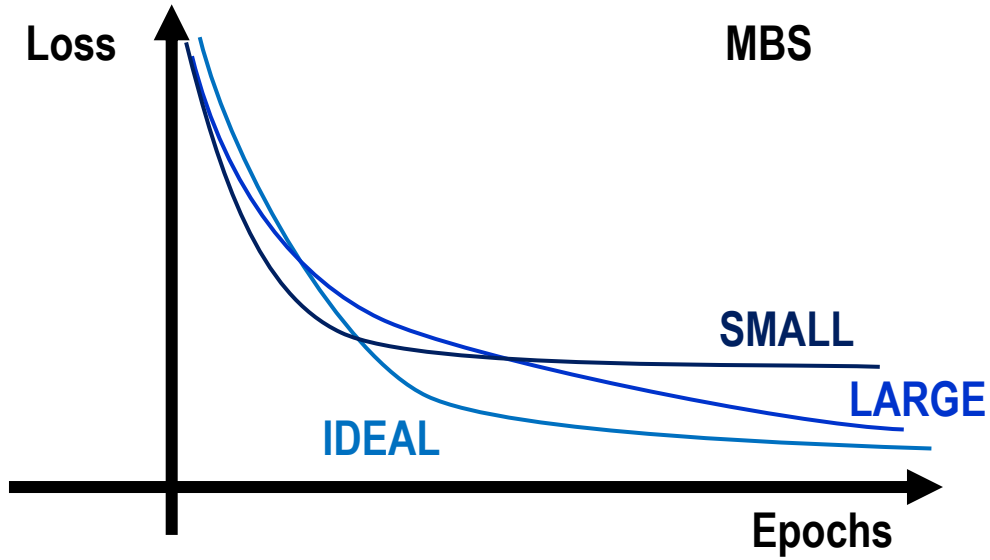
- **Accumulates prediction errors across all training examples**
- **Fewer updates:**
  - More computationally efficient, however, it requires the entire training dataset in memory
  - More stable error in the gradient → might result in a more stable convergence. However, it might find a local minimum.
- **Can be easily parallelised (separating the calculation of prediction errors and model update)**

Step 2: Optimization setup

(M-BGD) **Mini-Batch** gradient descent: batch size is: >1 sample, <all the training samples

- More frequent model update than BGD: should avoid local minima
- Error information is accumulated across mini-batches (as in BGD)

  - What **Mini-Batch Size (MBS)**?
    - Small MBS: learning process converges quickly at the cost of noise in the training process.
    - Large MBS: learning process converges slowly with accurate estimates of the error gradient.
    - **Sizes: 32, 64, and 128 (power of two that fits the memory requirements of the GPU or CPU hardware).**
      Note: the final batch might have fewer samples than other batches. Alternately, you can remove some samples from the dataset or change the batch size such that the number of samples in the dataset does divide evenly by the batch size.
  - How to tune the mini-batch size for the specific case?
    - **check learning curves of loss vs epochs → trade-off between convergence and performance**



Step 2: Optimization setup

NN require randomness when being fit to a dataset: weight initialization & random data shuffling

- **Weight initialization**: embracing randomness to start the search for the "optimal" set of weights prior to training.
  - set the weights to **small random values** that define the starting point for the optimization to avoid getting stuck in local minima
  - each input to a node will have a different weight initialization → to obtain a different output from each input
  - a neural network is initialized with a different set of weights → determining differences in optimization results for each batch
  - the choice of a particular initialization model depends on the nonlinear **activation functions for hidden layers**

- **Random shuffling of data** during each epoch → which results in differences in the gradient estimate for each batch.

Perhaps the only property known with complete certainty is that the initial parameters need to "break symmetry" between different units. If two hidden units with the same activation function are connected to the same inputs, then these units must have different initial parameters. If they have the same initial parameters, then a deterministic learning algorithm applied to a deterministic cost and model will constantly update both of these units in the same way.

Page 301, Deep Learning, 2016.

Note: initialize the random number generator **with a fixed seed value**, so that you can run the same code again and obtain the same result. Useful to: demonstrate a result; compare algorithms; debug a part of your code.

Step 2: Optimization setup

Need to calculate the **gradients** and set the **learning rate**!

- The **backpropagation** can be used to compute the gradients. From the loss (obtained in forward propagation), calculate the partial derivatives, backpropagating the errors **from the last to the first layer of the network**!*
- Typically, this is implemented using AutoDiff (covered in the next lecture)

$$w_{ij}(t) = w_{ij}(t-1) - \alpha \frac{\partial L(\mathbf{w}, b)}{\partial w_{ij}}$$

$$b(t) = b(t-1) - \alpha \frac{\partial L(\mathbf{w}, b)}{\partial b}$$

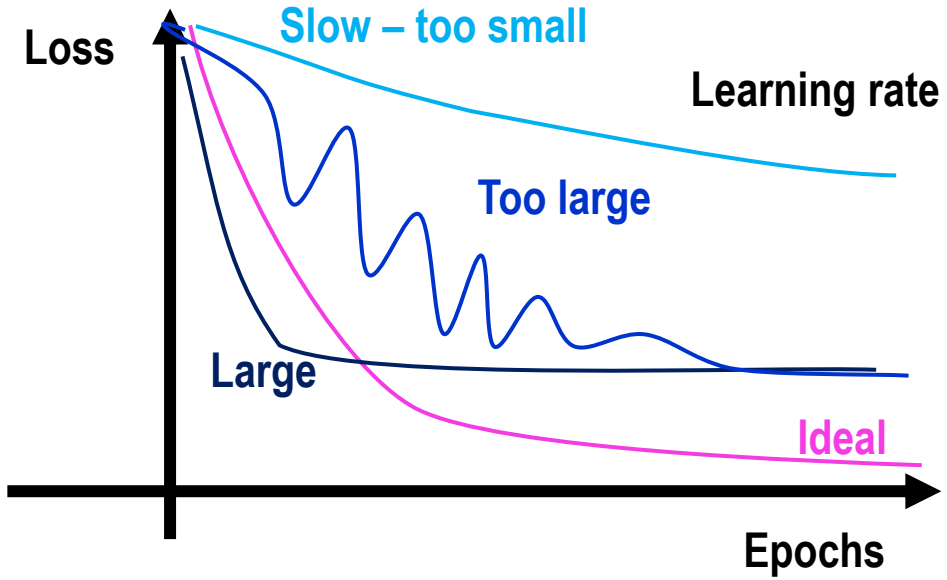
- Weights are not updated by the full amount.
- The learning rate (a small positive value, often in the range between 0.0 and 1.0) is the amount that the weights are updated during training: can be **constant or decreasing with time (using a step, linear or exponential decay)**.

The learning rate is perhaps the most important hyperparameter. If you have time to tune only one hyperparameter, tune the learning rate.
Page 429, Deep Learning, 2016.

*An intuitive explanation on backprop for MLP: https://www.youtube.com/watch?v=Ilg3gGewQ5U&t=70s

Step 2: Optimization setup

Diagnostic on the learning rate: plot loss over training epochs during training.



- **Learned too quickly**: sharp variation and plateau.
  A large learning rate: the model learn faster, at the cost of arriving on a sub-optimal final set of weights and biases.
  Too large? weight and biases updates that will be too large and the loss will oscillate over training epochs.

- **Learning too slowly**: little or no change.
  A small learning rate: may take significantly longer to train to learn a more optimal or even globally optimal set of weights and biases.
  Too small? may never converge or get stuck in a suboptimal solution.

Note: SGD uses a single learning rate for all weight updates and the learning rate does not change during training. Configuring the learning rate can be challenging and time-consuming → adaptive learning rate approaches… such as ADAM (see backup slides!)

Steps for building a fully connected Neural Network model

Step 1: NN architecture setup

Step 2: Optimization setup

Step 3: Network training

Step 4: Tuning of Network parameters on validation data set

Step 5: Test on the NN

Step 4: Tuning of Network parameters on validation data set

Adjusting **model configuration** (i.e. hyperparameters) using the validation dataset

Start from the model identified in step 1-3.
Note: only the training dataset was used up to now!

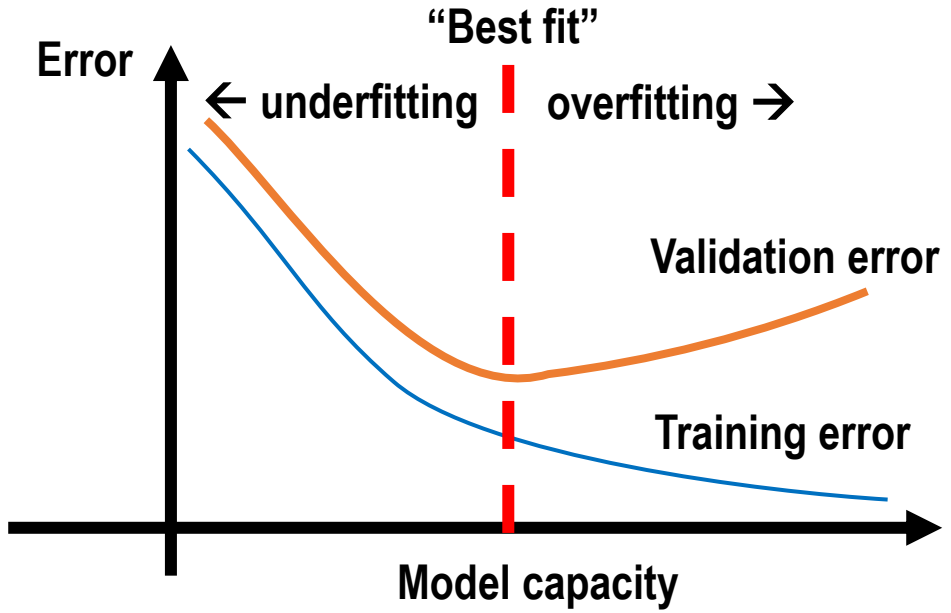
Goal: We want to explore different model complexity to find a model that is not **underfitting or overfitting the data!**

The validation dataset is used in this step!

Hyperparameters to be tuned

(i) number of nodes of the hidden layer;
(ii) number of hidden layers;
(iii) activation function for the hidden layers.

Searching across all the possible hyperparameters configuration can be more an art than a science!!



Note: **the loss/error will never be zero** (only for very trivial machine learning problems) –there will always be some error.
Goal: choose a model configuration and training configuration that achieve the **lowest loss** and highest **accuracy possible for a given dataset**.

Step 4: Tuning of Network parameters on validation data set

Given a set of hyperparameters, we assess the **loss and accuracy on both on the training dataset** (where the optimization setup remains unchanged from step 3) **and on the validation dataset**

```
Epoch 145/150
514/514 [==============================]
- 0s - loss: 0.5252 - acc: 0.7335 - val_loss: 0.5489 - val_acc: 0.7244
```

The historical **data of loss & accuracy over epochs can be used to investigate**:

- Speed of convergence over epochs (slope)
- Whether the model may have already converged (plateau of the line)
- over-learning the training data (inflection for validation line)
- ….

Metrics for model selection: lowest **loss** and highest **accuracy** for a given dataset

PSEUDO code

```
# split data
data = ...
train, validation, test = split(data)

# tune model hyperparameters
parameters = ...
for params in parameters:
        model = fit(train, params) - step 3
        skill = evaluate(model, validation) - step 4
Return: loss and accuracy for each model
```

Adapted from:
https://machinelearningmastery.com/difference-test-validation-datasets/

Note: a good model is said to have skill, meaning that its predictions are better than random chance.

Step 4: Tuning of Network parameters on validation data set

Popular **hyperparameters search strategies**, tips and strategies for diagnosis

Popular search strategies:

- **Random**: random configurations of (i)-(iii).
- **Grid**: systematic search across (i)-(iii).
- **Heuristic**: directed search across configurations (e.g. genetic algorithm, Bayesian optimization).
- **Exhaustive**: consider all combinations of (i)-(iii) → feasible only for small networks and datasets.

Tips:

- bound the size of search space (especially of number of hidden layers!);
- fit models to smaller portions of dataset.

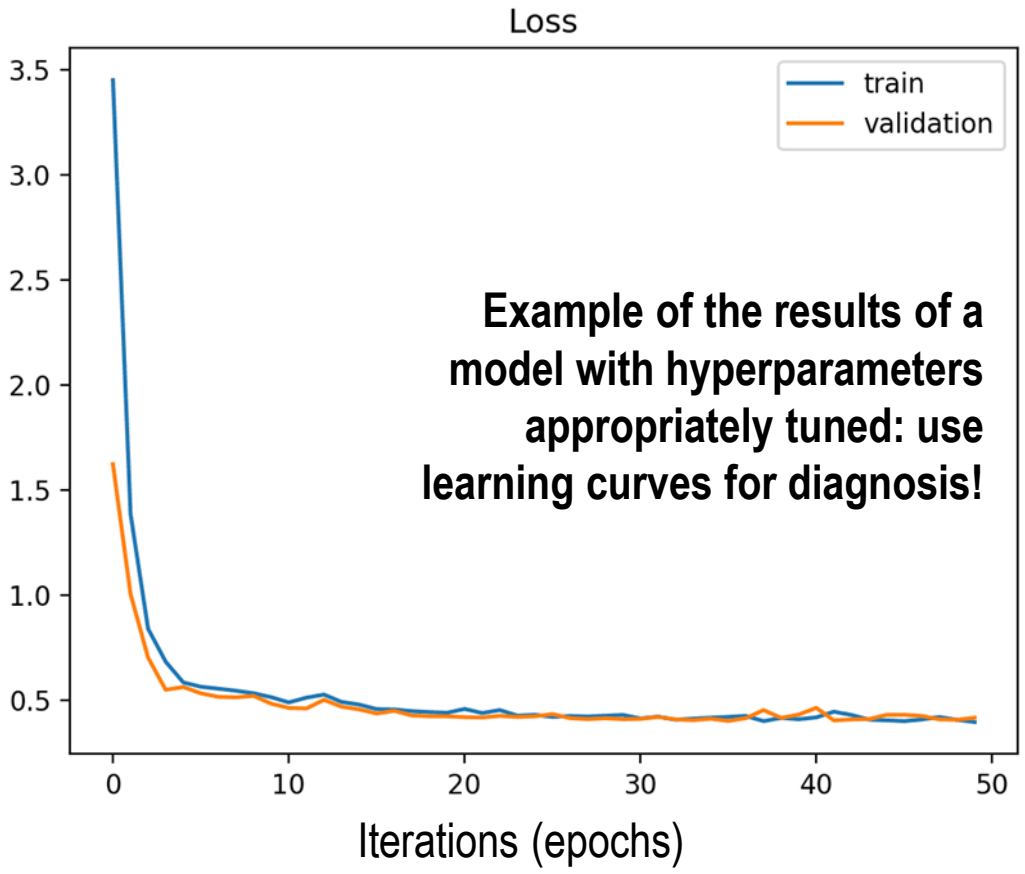


Figure from: machinelearningmastery.com

Steps for building a fully connected Neural Network model

Step 1: NN architecture setup

Step 2: Optimization setup

Step 3: Network training

Step 4: Tuning of Network parameters on validation data set

Step 5: Test on the NN

Step 5: Test on the NN

- The training configuration and the model configuration are used to identify the **"best fit" model**

- **Generalization performance**: model ability to perform well on previously unobserved inputs, that is the model prediction performance on unseen data (holdout)

- The test data set (NOT USED DURING TRAINING OR VALIDATION!) is used to assess this model (unbiased) performance by comparing the model predictions with the targets

Using unseen data to assesses the **generalization performance**

PSEUDO code

```
# split data
data = ...
train, validation, test = split(data)

# tune model hyperparameters
parameters = ...
for params in parameters:
        model = fit(train, params)
        skill = evaluate(model, validation)

# evaluate final model for comparison with other models
model = fit(train)
skill = evaluate(model, test)
```

From: https://machinelearningmastery.com/difference-test-validation-datasets/

Step 5: Test on the NN

Metrics: function used to judge the performance of a model.

"Metric functions are similar to loss functions, except that the results from evaluating a metric are not used when training the model. Note that you may use any loss function as a metric."

Accuracy metrics

- Accuracy class
- BinaryAccuracy class
- CategoricalAccuracy class
- SparseCategoricalAccuracy class
- TopKCategoricalAccuracy class
- SparseTopKCategoricalAccuracy class

Probabilistic metrics

- BinaryCrossentropy class
- CategoricalCrossentropy class
- SparseCategoricalCrossentropy class
- KLDivergence class
- Poisson class

Regression metrics

- MeanSquaredError class
- RootMeanSquaredError class
- MeanAbsoluteError class
- MeanAbsolutePercentageError class
- MeanSquaredLogarithmicError class
- CosineSimilarity class
- LogCoshError class

Classification metrics based on True/False positives & negatives

- AUC class
- Precision class
- Recall class
- TruePositives class
- TrueNegatives class
- FalsePositives class
- FalseNegatives class
- PrecisionAtRecall class
- SensitivityAtSpecificity class
- SpecificityAtSensitivity class

Image segmentation metrics

- MeanIoU class

Hinge metrics for "maximum-margin" classification

- Hinge class
- SquaredHinge class
- CategoricalHinge class

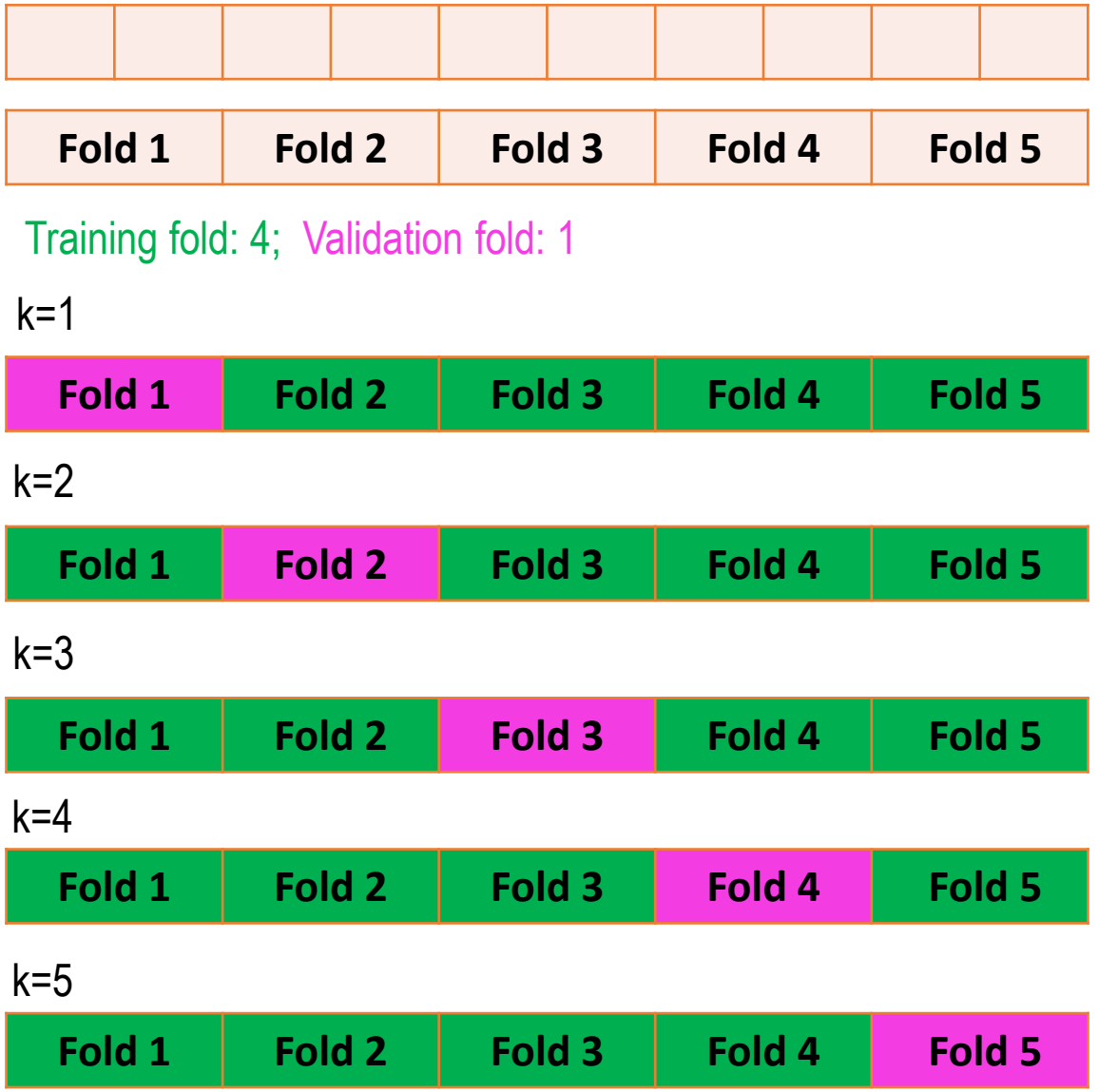
From: https://keras.io/api/metrics/

Applied machine learning: **k-fold cross-validation (to reduce model variance)**

- Common practice to tune model hyperparameters using **k-fold cross-validation**
  - **Reference to a "validation dataset" disappears**, and the data is **split between training and validation fold**.
  - The training dataset is split into **k subsets**,
  - **k models are trained** on the k-1 subsets
  - Each model performance is evaluated on the **held-out validation fold.**
  - The process is repeated until all subsets are given an opportunity to be the held-out validation set.
  - **The performance measure is then averaged across all models that are created (mean_skill).**
  - **A standard deviation can be computed to get an idea of the average spread of scores around the mean_skill:**

- The folds are **stratified**: balanced number of instances of each class in each fold.
- Cross-validation is often NOT used in deep learning because of the computational cost

Dataset (test data holdout, NOT included below):
10 samples, k subsets = 5, each fold has 2 samples



Applied machine learning: **k-fold cross-validation - Pseudocode**

- Common practice to tune model hyperparameters using **k-fold cross-validation**
  - **Reference to a "validation dataset" disappears**, and the data is **split between training and validation fold**.
  - The training dataset is split into **k subsets**,
  - **k models are trained** on the k-1 subsets
  - Each model performance is evaluated on the **held-out validation fold.**
  - The process is repeated until all subsets are given an opportunity to be the held-out validation set.
  - **The performance measure is then averaged across all models that are created (mean_skill).**
  - **A standard deviation can be computed to get an idea of the average spread of scores around the mean_skill:**

- The folds are **stratified**: balanced number of instances of each class in each fold.
- Cross-validation is often NOT used in deep learning because of the computational cost

```
# split data
data = ...
train, test = split(data)

# tune model hyperparameters
parameters = ...
k = ...
for params in parameters:
        skills = list()
        for i in k:
                fold_train, fold_val = cv_split(i, k, train)
                model = fit(fold_train, params)
                skill_estimate = evaluate(model, fold_val)
                skills.append(skill_estimate)
        skill = summarize(skills)

# evaluate final model for comparison with other models
model = fit(train)
skill = evaluate(model, test)
scores.append(skill)
mean_skill = sum(scores) / count(scores)
standard_deviation = sqrt(1/count(scores) * sum( (score - mean_skill)^2 ))
```

Applied machine learning: **regularization methods -** modification of the learning algorithm **to reduce generalization error** (not the training error)

> Unless your training set contains tens of millions of examples or more, you should include some mild forms of regularization from the start.
>
> Page 426, Deep Learning, 2016.

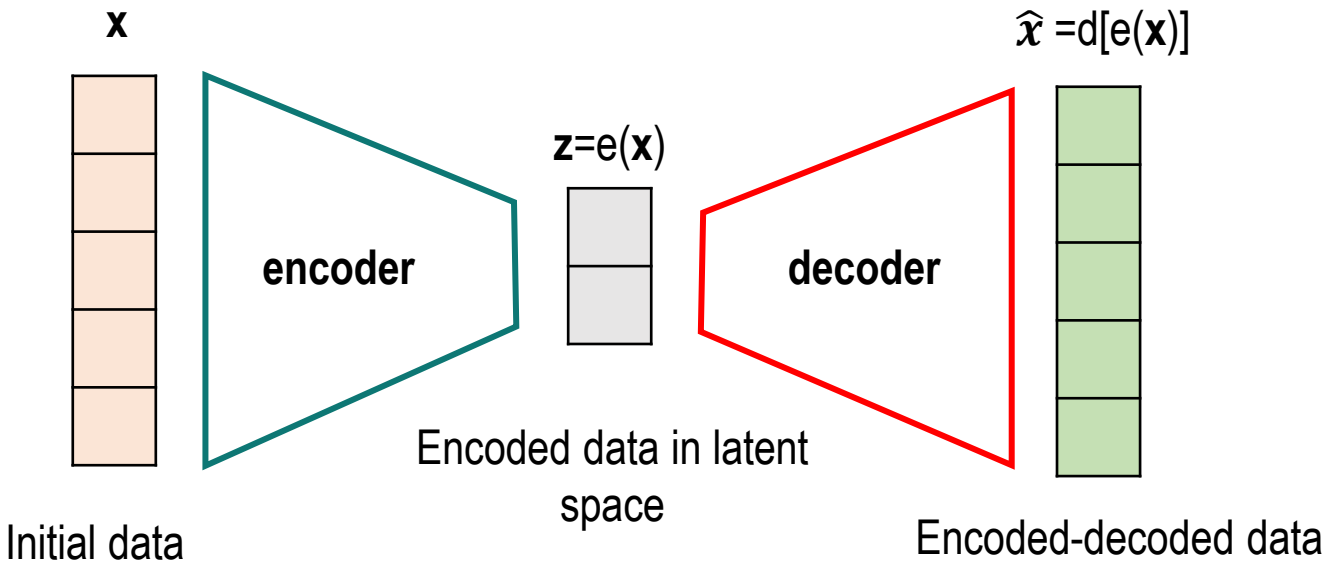
Most common strategies:
- Adding a penalty to the loss function
- **Weight Regularization (weight decay):** penalizes the model during training based on the magnitude of the weights.

Alternative methods (during training) for which it has been demonstrated (or proven) to approximate the effect of adding a penalty to the loss function:

- **Activity Regularization**: Penalize the model based on the magnitude of the activations.
- **Weight Constraint**: Set a range for the magnitude of weights.
- **Dropout**: Probabilistically remove inputs.
- **Noise**: Add statistical noise to inputs.

Early Stopping: Very popular alternative; Monitor model performance on a validation set and stop training when performance degrades.

SOTA architectures based on ANN: **AutoEncoder (AE) for dimensionality reduction**



$\hat{x}$=d[e(x)]=x Lossless encoding

Or **lossy encoding** otherwise

Loss=ll $\hat{x} - x$ll^2

Dimensionality reduction method based on finding the best encoder/decoder pair among a given family (typically deep ANN) so that

- keeps the **maximum of information** when encoding
  → dimension of the latent space should keep **the "main structure"** existing among data
- has the **minimum reconstruction** error when decoding

lack of interpretable and exploitable structures in the latent space

The AE can be used to generate **new data** by decoding points that are randomly sampled from the latent space! → quality and relevance of generated data depend on the **regularity** of the latent space

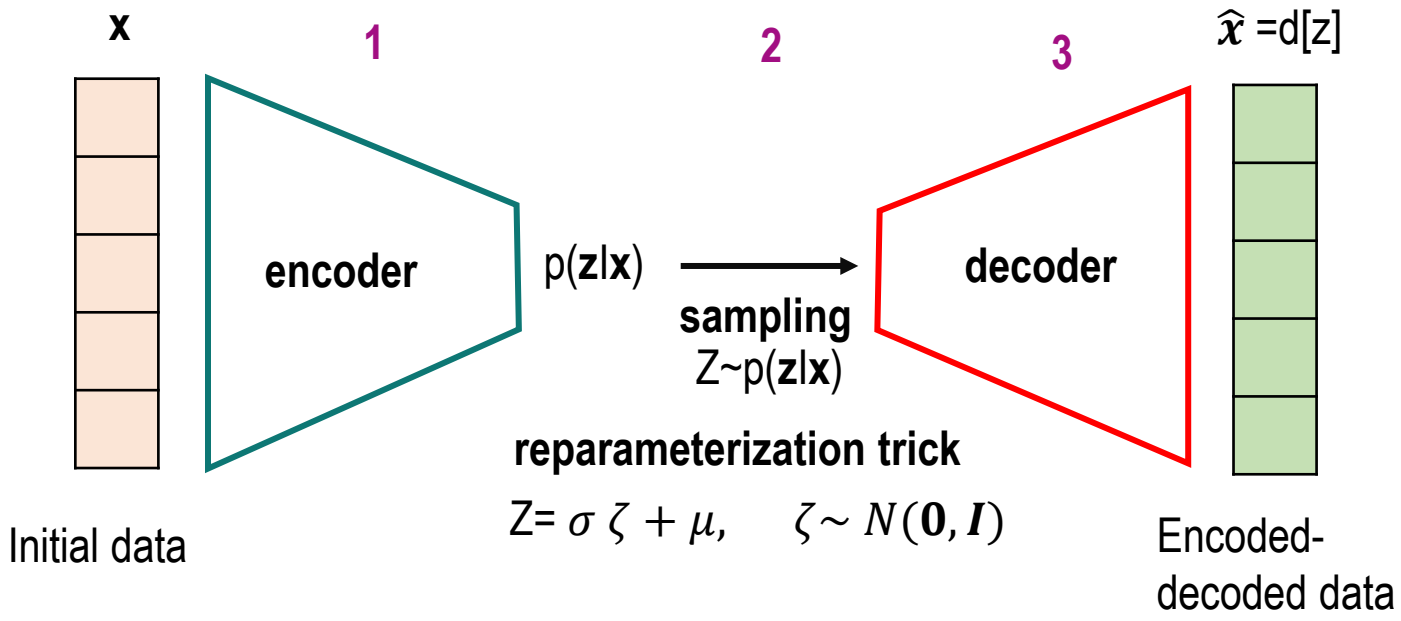
SOTA architectures: **Variational AutoEncoder (VAE) for generating new data**

Introduces some regularisation of the latent space: instead of encoding an input as a single point

→ **the input is encoded as a distribution over the latent space**

Steps:

1. the input is encoded as distribution over the latent space
2. a point from the latent space is sampled from that distribution
3. the sampled point is **decoded** and the reconstruction error can be computed
4. the reconstruction error is backpropagated through the network



- The encoded distributions **are chosen to be normal** so that the encoder can be trained to return the mean μ and the covariance matrix!
- The Loss is regularised by adding of the Kulback-Leibler divergence between the returned distribution and a standard Gaussian

Loss=ll $\hat{\boldsymbol{x}} - \boldsymbol{x}$ll^2 + $KL(N(\boldsymbol{\mu}, \boldsymbol{\sigma})||N(\mathbf{0}, \boldsymbol{I}))$

Today you learned about

- The elements of a **Perceptron** and of a Multi-layer Perceptron
- The 5 steps needed to setup a **Fully Connected Neural Network**
- Practices in applied machine learning: **k-fold cross-validation and regularization**
- SOTA architectures based on ANN: **AutoEncoders** and **Variational AutoEncoders**

Next Lecture

Neural Network architectures for dataset made of images:

- Convolution Neural Network (CNN)
- U-net
- Residual Neural Network (ResNet)

Check Bonus slides!

The material presented in based on

- **Deep Learning: foundations and concepts, C. Bishop, H. Bishop** (https://www.bishopbook.com/)
- **Pattern Recognition and machine learning, C. Bishop** (https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf)
- **Deep Learning, I. Goodfellow, Y. Bengio, A. Courville** (https://www.deeplearningbook.org/)
- **Excellent online resources with coding examples**:
  - https://machinelearningmastery.com/

I attempted to credit all the content/figures where I could find the original source. Sometimes the material is so widely used that it's difficult to be sure of the original author. Please let me know if you find missing references.

Bonus slide: Things you should **never forget** when building your FCNN!

1. Data quality needs to be checked & data clearing must be carried out (see L1)
2. Careful data partition to avoid not representative training/validation/test datasets (see L1)

Data

3. Select the most appropriate initial architecture for the task at hand to have a suitable model capacity and to reduce computational cost.
4. Apply Deep Learning architectures ONLY IF YOU REALLY NEED THEM

Architecture setup

5. Initialize the random number generator with a fixed seed value
6. Weight initialization and data scaling depend on the activation function chosen
7. Use regularization method to reduce generalization error.
8. Carry out diagnostic on the learning rate and mini-batch size using loss vs epochs

Training and optimization

9. Hyperparameters search strategies work better when you bound the search space
10. Carry out diagnostic on model speed of convergence using loss & accuracy over epochs
11. When using k-fold cross-validation report mean and standard deviation of the performance metric

Validation

12. Carefully choose the metric to assess the model generalization performance

testing

Step 2: Optimization setup

ADAM (ADAptive Moment estimation): an efficient alternative to SGD (*shown for **w only**)

ADAM computes **adaptive learning rates** for each parameter (the weight!) at the end of each iteration t:

$$g_j(t) = \frac{\partial Loss(\mathbf{w}(t))}{\partial w_j(t)}$$

1. **Computing the Gradient (g) of the loss function for a batch of data**.

2. **Updating Bias-Corrected First Moment Estimate (mean, m):** tracks a moving average of the past gradients (first moment), which decays exponentially (analogous to the Momentum optimizer). ADAM applies a bias correction. The exponential decay rate for the 1st moment estimate, **beta1**, is usually set to 0.9

$$m(t) = \beta_1 m(t-1) + (1-\beta_1) g_j(t)$$

$$\hat{m}(t) = m_{bias\_corrected}(t) = \frac{m(t)}{(1-\beta_1^t)}$$

3. **Update Bias-Corrected Second Moment Estimate (uncentered variance, v):** tracks a moving average of the squares of the gradients, which decays exponentially. This is used to scale the learning rate (similar to Adadelta and RMSProp). Another bias correction is applied to this estimate. The exponential decay rate for the 2nd moment estimate, **beta2**, is usually set to 0.999.

$$v(t) = \beta_2 v(t-1) + (1-\beta_2) g_j^2(t)$$

$$\hat{v}(t) = v_{bias\_corrected}(t) = \frac{v(t)}{(1-\beta_2^t)}$$

4. **Compute Parameter Update**:
Add a small constant **epsilon**= 1E-8 to avoid division by zero; set the learning rate or step size (**alpha**) to 0.001.

$$w_j(t) = w_j(t-1) - \alpha \frac{\hat{m}(t)}{\sqrt{\hat{v}(t)} + \varepsilon}$$

The adaptive learning rate can be explicitly written as:

$$\hat{\alpha} = \alpha \frac{\sqrt{(1-\beta_2)}}{(1-\beta_1)} \quad \Rightarrow \quad w_j(t) = w_j(t-1) - \hat{\alpha} \frac{m(t)}{\sqrt{v(t)} + \varepsilon}$$